Computing with Time

Distributed Systems

Andrea Omicini
andrea.omicini@unibo.it

Dipartimento di Informatica – Scienza e Ingegneria (DISI)
Alma Mater Studiorum – Università di Bologna

Academic Year 2025/2026

Next in Line...

Computing Needs Time

Computation & physical processes[Lee, 2009]

- on the one hand, most computational devices nowadays are not just general-purpose computers
  - e.g., cars, medical devices, instruments, communication systems, industrial robots, toys, games, ...
  - the explosion of the Internet of things (IoT) asks them to become more and more intelligent and networked[Atzori et al., 2010, Gubbi et al., 2013]
  - *special-purpose computers*, yet with computational architecture and power more and more *similar* to general-purpose computers
  - with their increased complexity possibly at the expense of their *dependability*
- on the other hand, general-purpose computers are more and more required to interact with physical processes
  - they integrate media, such as video and audio
  - their migration to personal devices and pervasive systems asks them to *sense* physical dynamics and *control physical devices*

Cyber-Physical Systems (CPS) I

Time, computation & CPS[Lee, 2009]

- the foundations of computing are still rooted in Turing,[Turing, 1937] Church, and von Neumann[Burks et al., 1982]
  - they are about the *transformation of data*
  - they do *not* deal with the *dynamic* of *physical processes*
- cyber-physical systems (CPS) emerge from the integration of physical *systems* and *processes* with *networked computing*
  - CPS deeply embed computations and communication interacting with physical processes to add new capabilities to physical systems

! CPS are about merging computing and networking with physical systems to create new science, techniques, and products

Cyber-Physical Systems (CPS) II

Remarkable example of CPS

- high-confidence medical devices
- assisted living
- traffic control and safety
- advanced automotive systems
- process control
- energy conservation
- environmental control
- avionics, instrumentation
- critical infrastructure control—electric power, water resources, communications systems
- distributed robotics—telepresence, telemedicine
- defence systems
- manufacturing
- smart structures

Cyber-Physical Systems (CPS) III

The passage of time

- the technological basis that engineers and computer scientists have chosen for general-purpose computing and networking does *not support* well the applications dealing with physical processes
  - CPS in particular
- the passage of time is essentially *absent* in computing[Lee, 2009]

Misconceptions about Time in Computing [Lee, 2009] I

computing takes time — it is not just the fact that efficiency has its limits: it is the fact that time is always abstracted away from computing, so that *computing* always *omits time*

time is a resource — yes, but a different sort of resource: it is practically unbounded, and is expended anyway; while it is ok having generic ways to deal with resources in programming languages, *time* needs to be a *semantic property*

time is a nonfunctional property — with Turing Machine, function can be computed by abstracting from time: however, the function of a computation in a CPS is defined through *actions*, occurring in *physical time*, with a *duration*

Misconceptions about Time in Computing [Lee, 2009] II

real time is a quality-of-service (QoS) problem — time in CPS is mostly not about efficiency: it is rather about *predictability* and *repeatability*; precision and variability in timing are QoS issues, whereas time itself is much more than that: time should be part of the semantics of programs

Computing & Time

core computing abstractions should be re-thought to include time

Next in Line...

Which Notion of Time?

Time

http://www.britannica.com/science/time

> *Time*, *a measured or measurable period, a continuum that lacks spatial dimensions. Time is of philosophical interest and is also the subject of mathematical and scientific investigation. . . .*

- what about our common sense notion of time?

Time after Relativity[Einstein, 1920]

No such a thing as *one time*[Rovelli, 2017]

- there is no longer a notion of *true time*
- there is no longer a notion of *unique time*
- physics no longer describes how things evolve over time, rather
  - how *things* evolve in *their own times*
  - how *times* evolve one with respect to *each other*

Moreover[Rovelli, 2017]

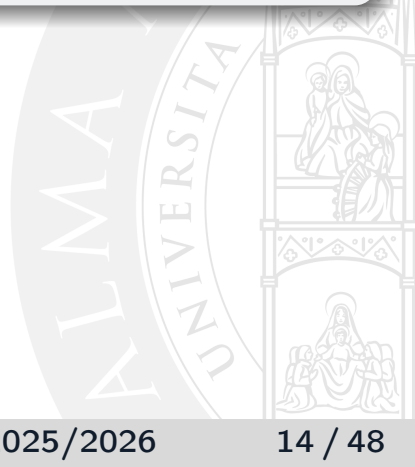
- there is no longer a *direction*
- there is no longer a *present* time, a "now"
- time is not *independent*
- time is just related to *change*: *events* happens, and time can be used to *relate* their *dynamics*

Next in Line...

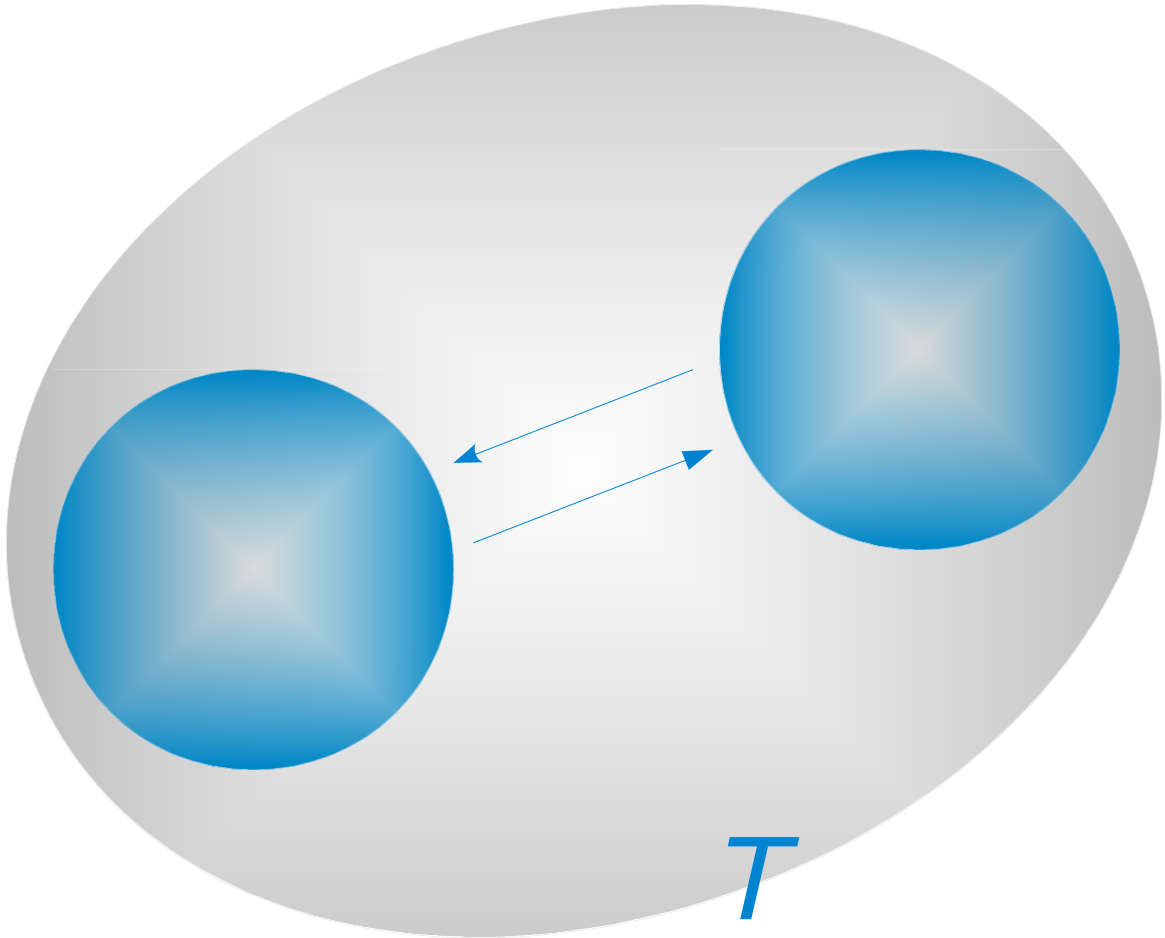
Parallel, Concurrent, Distributed Systems: Recap I

Parallel computing & systems

- given a computational system, we talk of parallel computation whenever the *temporal* context is the same for all computational process
- a parallel system is a computational system performing parallel computations



Parallel, Concurrent, Distributed Systems: Recap II



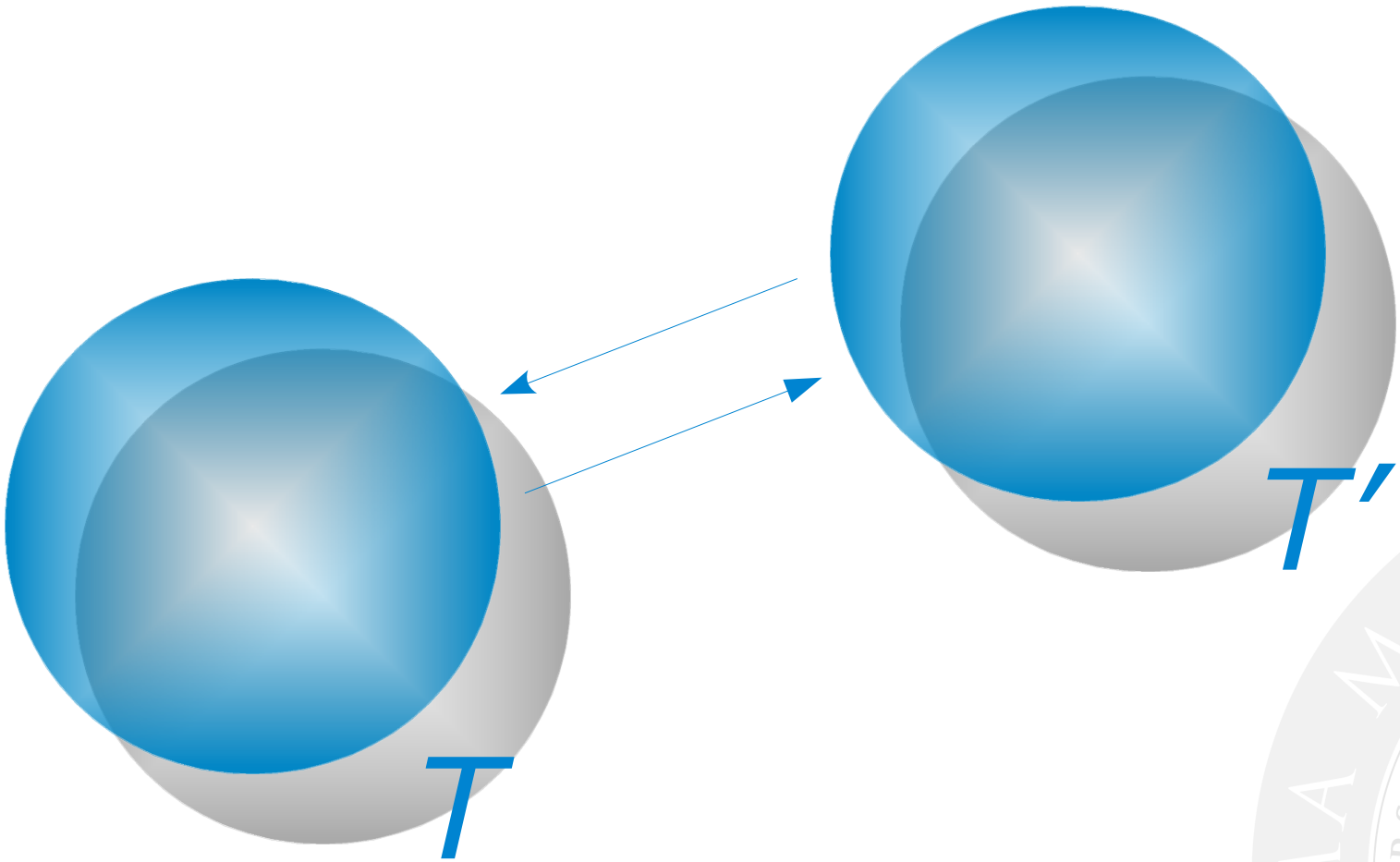
Parallel computing: the same temporal context T for all processes

Parallel, Concurrent, Distributed Systems: Recap III

Concurrent computing & systems

- given a computational system, we talk of concurrent computation whenever at least two computational processes have a different *temporal* context
- a concurrent system is a computational system performing concurrent computations

Parallel, Concurrent, Distributed Systems: Recap IV



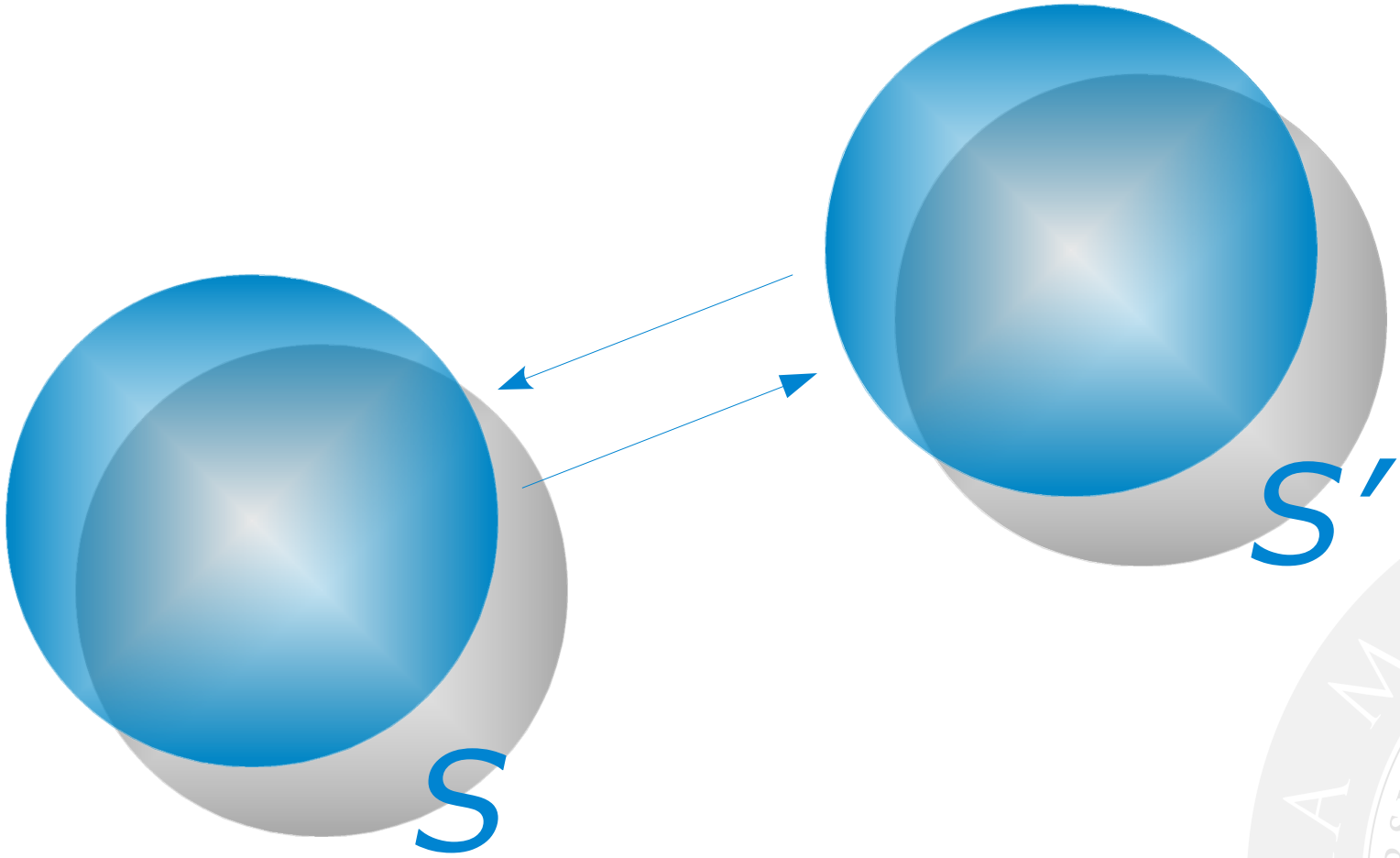
Concurrent computing: different temporal contexts $T \neq T'$ for different processes

Parallel, Concurrent, Distributed Systems: Recap V

Distributed computing & systems

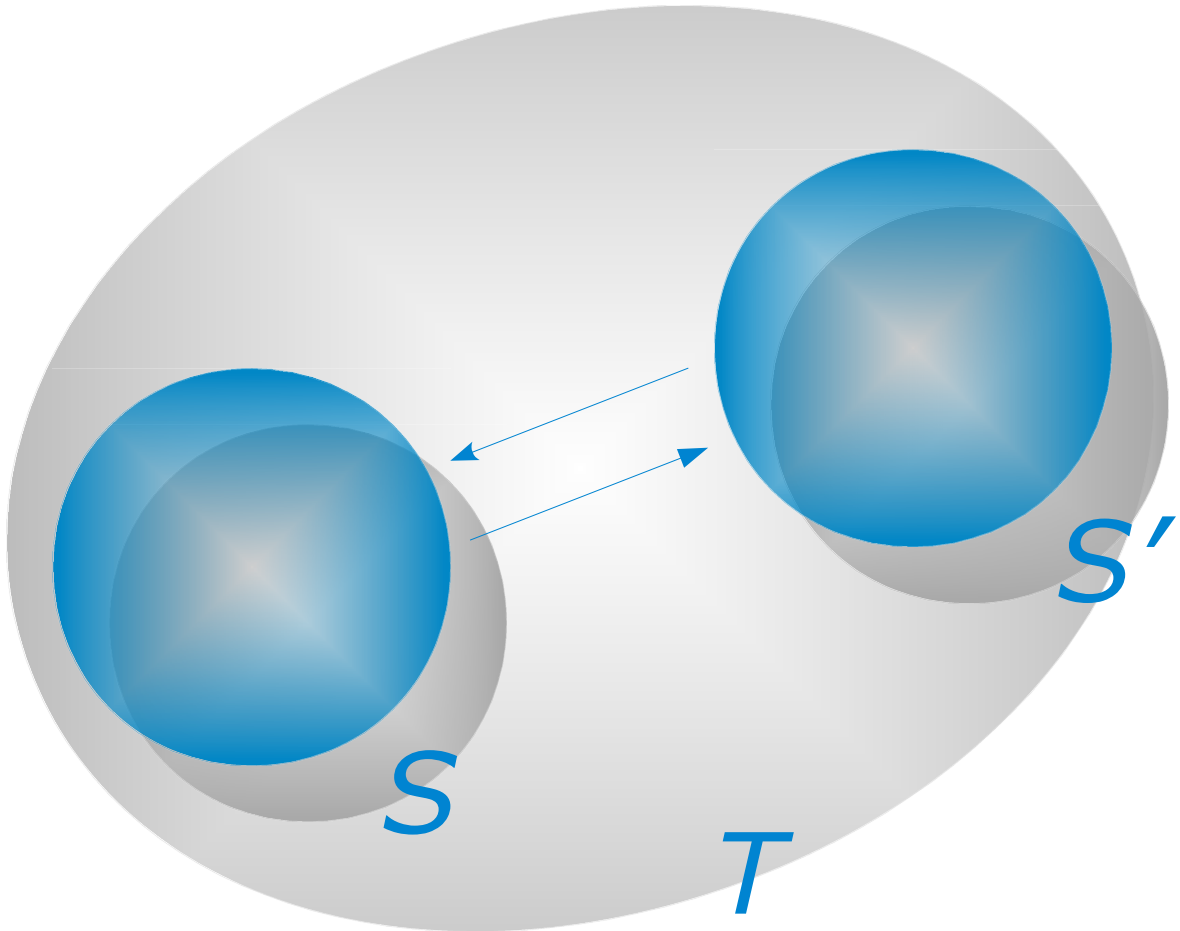
- given a computational system, we talk of distributed computation whenever at least two computational processes have a different *spatial* context
- a distributed system is a computational system performing distributed computations

Parallel, Concurrent, Distributed Systems: Recap VI



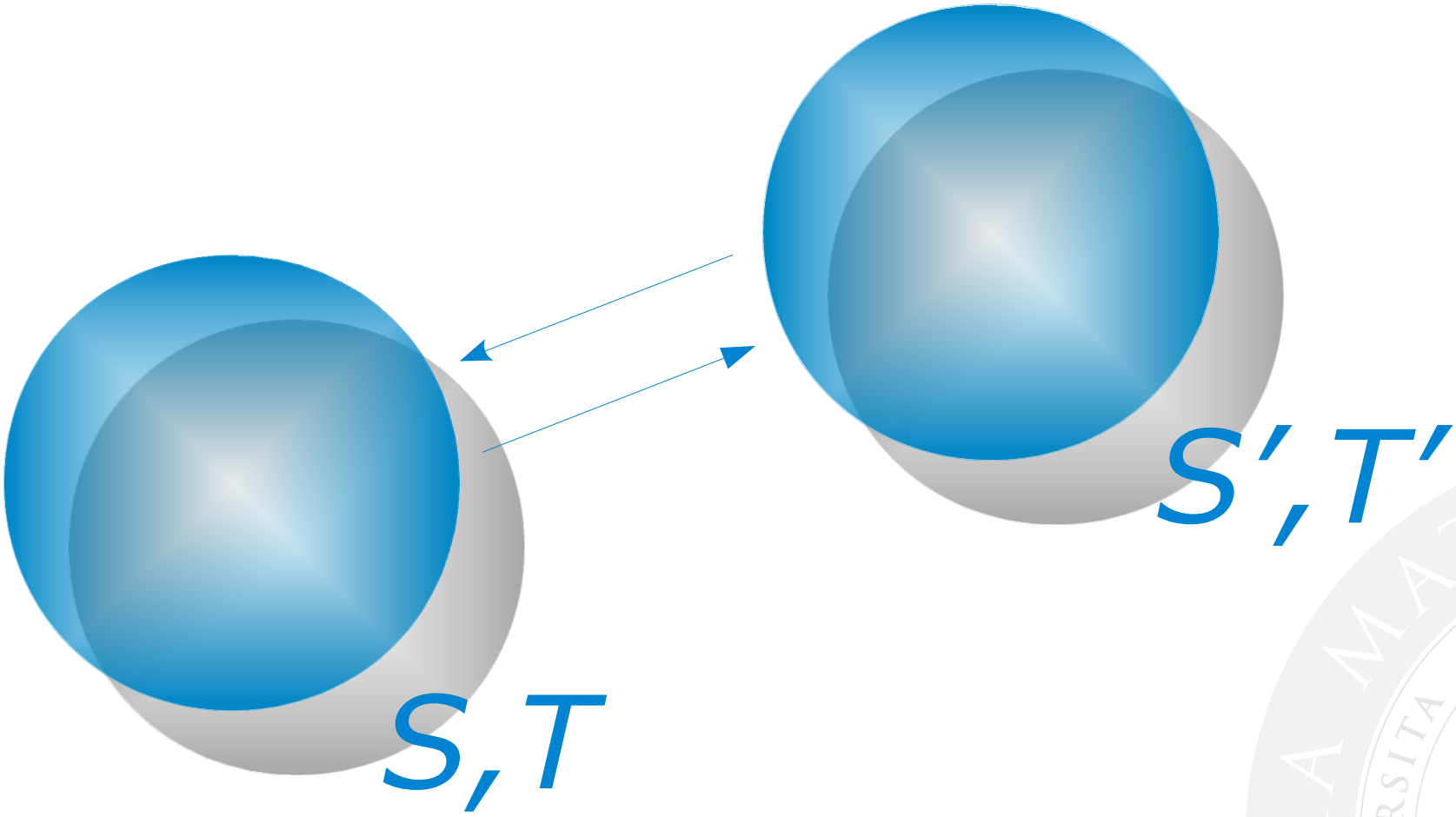
Distributed computing: different spatial contexts $S \neq S'$ for different processes

Spatial vs. Temporal Contexts: Recap I



Distributed parallel computing: $S \neq S'$, same T

Spatial vs. Temporal Contexts: Recap II



Distributed concurrent computing: $S \neq S'$, $T \neq T'$

Basic Questions

- is there any useful and well-founded notion of *time* in a distributed system?
- is there any coherent notion of *global* time in a distributed system?
- if not, what can we do about this?
- if not, how can we *synchronise* activities within a distributed system?

Synchronous

http://www.oxforddictionaries.com/definition/english/synchronous

synchronous,

1. *Existing or occurring at the same time*

   ...

The Issue of Time

Time in distributed systems

- in *centralised*, non-distributed systems, time is unambiguous
- in a distributed system, there is no *natural* notion of time
  - distributed concurrent computing is the most natural and general computational model for distributed systems

? is it *possible* to build up a global notion of time in any distributed system?

? is it *useful* to build up a global notion of time in any distributed system?

Focus on...

Physical Clocks[Kshemkalyani and Singhal, 2011] I

Timers

- a clock in a computer is actually a *timer*
  - typically, an oscillating quartz with a *counter* and a *holding register*
  - when the counter gets to zero, an interrupt is generated, and the counter is reloaded from the holding register
  - each interrupt is a clock tick

Physical Clocks[Kshemkalyani and Singhal, 2011] II

Physical clock synchronisation

- no need in centralised systems
  - generally, there is only a single clock
  - a process gets the time by issuing a system call to the kernel
  - when another process after that tries to get the time, it will get a higher time value

  → *total ordering of events*
- distributed systems have no global clock, no common memory
  - each process has its own internal clock and notion of time
  - each clock possibly *drifting* apart several seconds per day
  - even when synchronised (e.g., at start), different clock ticks could lead to differences in time

  → clock synchronisation needed

  ? . . . or, do we?

Physical Clocks[Kshemkalyani and Singhal, 2011] III

Definition

Physical clock synchronisation is the process of ensuring that *physically distributed processors* have a *common notion of time*

Why physical clock synchronisation?

For most applications and algorithms that run in a distributed system, we need to know time in the following contexts

- the time of the day at which an event happened on a specific machine in the network
- the time interval between two events that happened on different machines in the network
- the relative ordering of events that happened on different machines in the network

Definitions and Terminology [Kshemkalyani and Singhal, 2011] I

Given two machines a, b in a distributed system, with their clocks C_a, C_b, we define

time — function $C_a(t)$ gives the time of the clock for machine a

- $C_a(t) = t$ for a *perfect clock*

frequency — frequency is the *rate* at which a clock progresses

- C'_a is the *frequency* at time t of clock C_a
- (first) derivative w.r.t. to time of the clock C_a

offset — clock offset is the difference between clock and real time

- $C_a(t) - t$ is the *offset* of clock C_a
- $C_a(t) - C_b(t)$ is the *offset* of clock C_a relative to clock C_b at time t

Definitions and Terminology [Kshemkalyani and Singhal, 2011] II

skew — clock skew is the *frequency difference* between the clock time and the perfect clock

- $C'_a(t) - C'_b(t)$ is the *skew* of clock C_a relative to C_b at time t

drift — clock drift is $C''_a(t)$, i.e. the second derivative w.r.t. time of the clock

- $C''_a(t) - C''_b(t)$ is the *drift* of clock C_a relative to clock C_b at time t

Physical Clock Synchronisation: Global Absolute Time I

Physical clock synchronisation: How?

- to be synchronised, physical clocks need an accurate real-time standard
- e.g., Universal Coordinated Time (UTC)[ITU Radiocommunication Assembly, 2002]

Physical Clock Synchronisation: Global Absolute Time II

Absolute time

- absolute time is handled by the BIPM (Bureau International des Poids et Mesures) in Sèvres, France
- BIPM combines, analyses, and averages the official atomic time standards of member nations around the world to create a single, official *Universal Coordinated Time* (UTC)[ITU Radiocommunication Assembly, 2002]
- broadcasted as a short radio pulse (WWV) by NIST (National Institute of Standard Time) every UTC second, and by satellites providing UTC service[Nelson et al., 2005]
- if one machine in the system has access to an UTC service, an algorithm can be used that synchronises all machines based on this

Physical Clock Synchronisation: Global Absolute Time III

Example: NTP

- Network Time Protocol (NTP)
  - RFC 5905 for NTP v. 4
- a time server has the global absolute time, and other machines have to synchronise

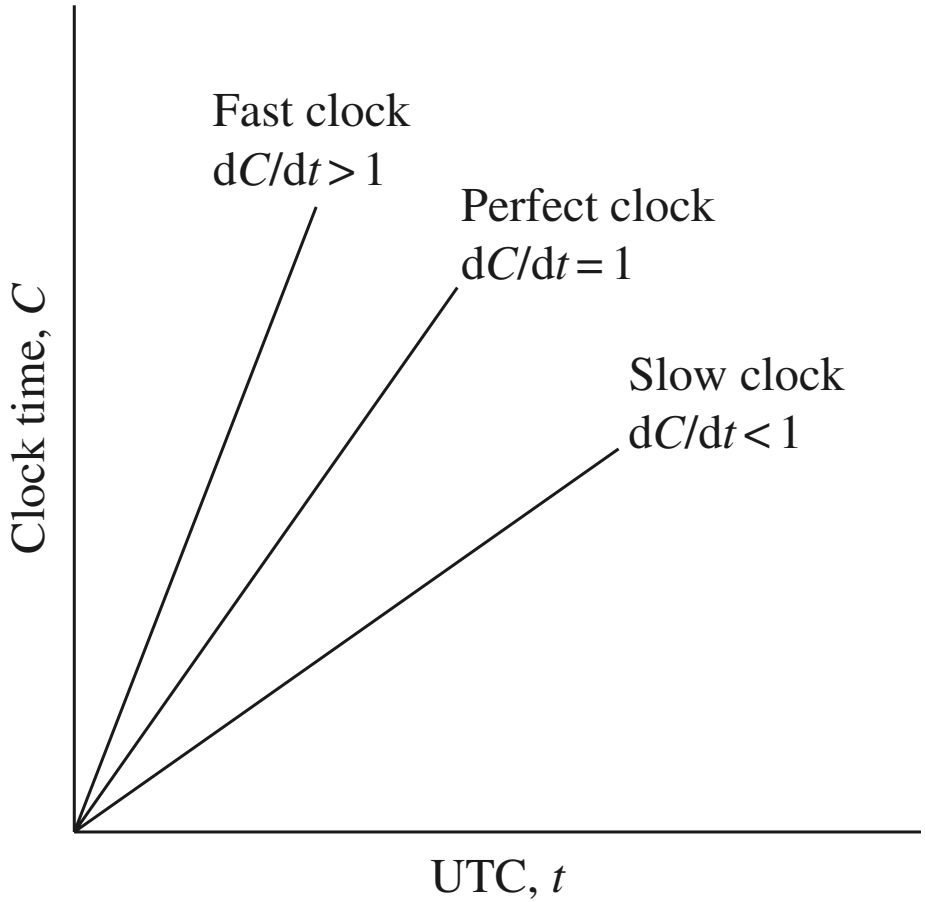
! clocks can only run *forward*—corrections cannot bring clocks backward

Clock Inaccuracies[Kshemkalyani and Singhal, 2011] I

- manufacturers typically specify the *maximum skew rate* ρ
- a timer (clock) is said to be *working within its specification* if

$$1 - \rho \leq \frac{dC}{dt} \leq 1 + \rho$$

Clock Inaccuracies[Kshemkalyani and Singhal, 2011] II



Fast, perfect, and slow clocks w.r.t. UTC

[Kshemkalyani and Singhal, 2011]

Absolute vs. Relative Time

- application needs may vary, and consequently mandate for different temporal properties
- or, a time server could not be available in a stable way, or even at all
- so, there are times when we actually need a common time related to physical time
  - global absolute time—e.g., UTC
- some other times, instead, we just need to share a common time, even unrelated to physical time
  - global relative time

Global Relative Time

Relative time

- sometimes, the only thing needed is that there is a *common time*, regardless of absolute time
- so, algorithms based on active servers polling other servers to find out the average time, and the required estimated corrections as well
- no machine is required to have UTC time

Examples

- the Berkeley Algorithm: time daemons in all machines poll and respond to each other, and agree on a common time[Gusella and Zatti, 1989]
- Reference Broadcast Synchronisation (RBS): global relative time in wireless networks[Elson et al., 2002]

Focus on...

Physical vs. Logical Time

Physical time not always needed

- till now, we have implicitly assumed that synchronisation is related to physical time
- however, we have also seen the case where the only actual requirement is *a shared notion of time* among the processes of a distributed system, with no need for it to be exactly the "real" time
- as a step further, we may observe that often the only need for a distributed system is a shared clock, even *unrelated* to real time
- a notion of logical time is both possible and useful

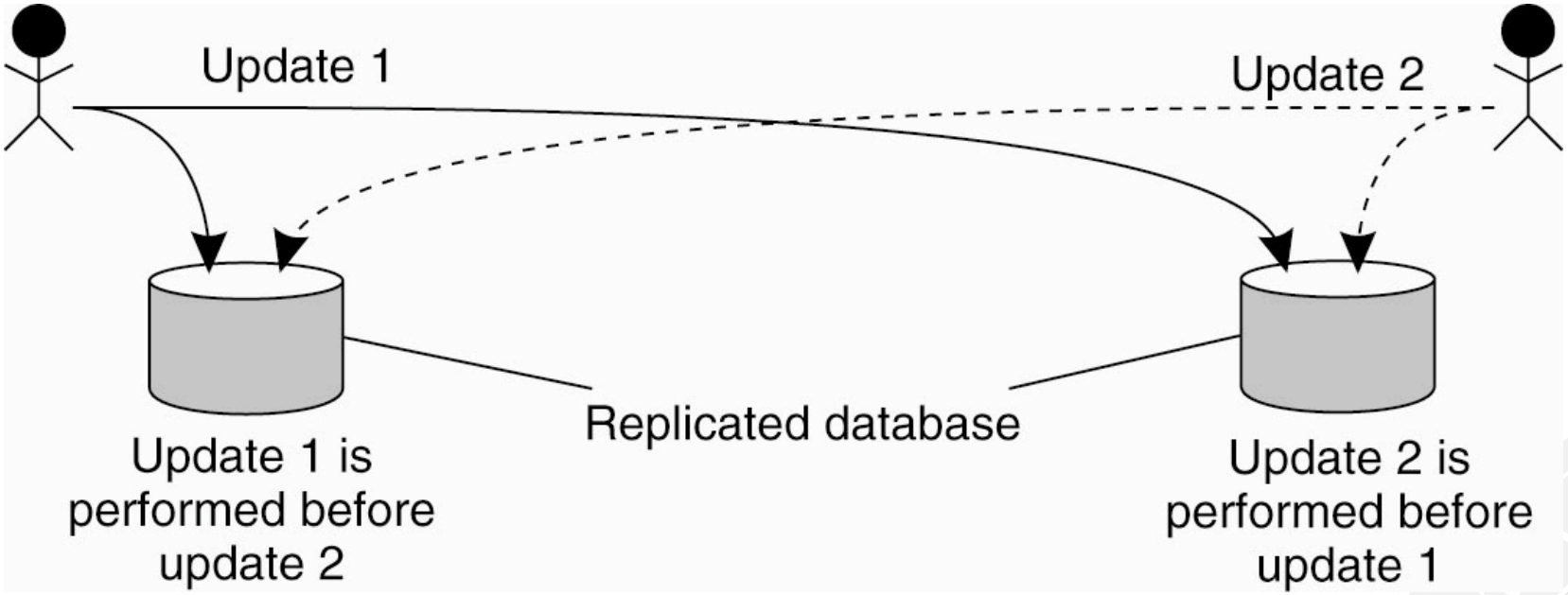
Example I

Problem

- a replicate database exists of the accounts of a bank in LA and NY
- a customer adds \$100 to his account, while at the same time a bank employee applies a 1% increment to the account
- given that the original account contained \$1000, it may easily happens that, say, the LA replica records \$1110, the NY one \$1111

→ inconsistency due to concurrent updates over a distributed replicated database

Example II



Inconsistency in a replicated database after two concurrent updates

[Tanenbaum and van Steen, 2017]

Logical Clocks[Lamport, 1978]

Synchronisation is possible with no need to be absolute

- if two processes do *not* interact, there is no need of synchronisation—lack of synchronisation would not be observable
- often, what really matters is not the exact time when events occur, but the *order* in which events occur
- example: UNIX `make`, Gradle

Logical clocks

- synchronisation of non-physical, logical clocks[Lamport, 1978] is then both *admissible* and *useful*

Next in Line...

Communication & Interaction in Distributed System

Communication is just half of the story

- interaction is a more general issue
- governing (inter)action is a fundamental issue in (distributed) systems
- doing the right thing at the right time is essential
- "at the right *time*" is the critical problem

Beyond Synchronisation I

Ordering events is not enough

- sometimes, more articulated policies are required
- for instance, to ensure that concurrent accesses to a shared resource could harm its consistency, or corrupt it

Mutual exclusion

- a number of algorithms—centralised, decentralised, distributed
  - e.g., Token Ring
- we do not review them here
- the main point: some of them are based on a coordinator, all of them are *coordination* algorithms

Beyond Synchronisation II

Election algorithms

- many distributed algorithms requires a *coordinator* to be elected
- again, we do not review them: election algorithms are (used by) coordination algorithms

It is not merely a matter of time

- synchronisation is about *when* things happen
- actions are more than sending messages
- interaction does not merely translate into suitably-ordered distributed actions—undifferentiated actions
- actions have a nature, and meaningful interaction within a distributed system typically depends on such a nature

Beyond Synchronisation III

The problem of coordination

- governing interaction based both on time, and on the nature of actions, and aimed at the achievement of some global objective for the distributed system
- this is the problem of *coordination* [Omicini et al., 2001]

Summing Up

Time & computation

- what is time?
- what is time in computational systems?

Time in distributed systems

- the issue of time
- physical time / clock
- logical time / clock
- causality and vector clocks

Toward coordination

- events have a nature: synchronisation is not enough

Computing with Time

Distributed Systems

Andrea Omicini
andrea.omicini@unibo.it

Dipartimento di Informatica – Scienza e Ingegneria (DISI)
Alma Mater Studiorum – Università di Bologna

Academic Year 2025/2026

References I

[Atzori et al., 2010] Atzori, L., Iera, A., and Morabito, G. (2010).
The Internet of Things: A survey.
Computer Networks, 54(15):2787–2805
DOI:10.1016/j.comnet.2010.05.010

[Burks et al., 1982] Burks, A. W., Goldstine, H. H., and von Neumann, J. (1982).
Preliminary discussion of the logical design of an electronic computing instrument.
In Randell, B., editor, *The Origins of Digital Computers*, Texts and Monographs in Computer Science, pages 399–413. Springer Berlin Heidelberg, Berlin, Heidelberg
DOI:10.1007/978-3-642-61812-3_32

[Einstein, 1920] Einstein, A. (1920).
Relativity: The Special and the General Theory.
Henry Holt and Company

[Elson et al., 2002] Elson, J., Girod, L., and Estrin, D. (2002).
Fine-grained network time synchronization using reference broadcasts.
ACM SIGOPS Operating Systems Review, 36(SI):147
DOI:10.1145/844128.844143

[Gubbi et al., 2013] Gubbi, J., Buyya, R., Marusic, S., and Palaniswami, M. (2013).
Internet of Things (IoT): A vision, architectural elements, and future directions.
Future Generation Computer Systems, 29(7):1645–1660
DOI:10.1016/j.future.2013.01.010

References II

[Gusella and Zatti, 1989] Gusella, R. and Zatti, S. (1989).
The accuracy of the clock synchronization achieved by TEMPO in Berkeley UNIX 4.3BSD.
IEEE Transactions on Software Engineering, 15(7):847–853
DOI:10.1109/32.29484

[ITU Radiocommunication Assembly, 2002] ITU Radiocommunication Assembly (2002).
Standard-frequency and time-signal emissions.
Recommendation ITU-R TF.460-6, International Telecommunications Union
https://www.itu.int/dms_pubrec/itu-r/rec/tf/R-REC-TF.460-6-200202-I!!PDF-E.pdf

[Kshemkalyani and Singhal, 2011] Kshemkalyani, A. D. and Singhal, M. (2011).
Distributed Computing: Principles, Algorithms, and Systems.
Cambridge University Press
https://www.cambridge.org/us/universitypress/subjects/engineering/communications-and-signal-processing/distributed-computing-principles-algorithms-and-systems

[Lamport, 1978] Lamport, L. (1978).
Time, clocks, and the ordering of events in a distributed system.
Communications of the ACM, 21(7):558–565
DOI:10.1145/359545.359563

[Lee, 2009] Lee, E. A. (2009).
Computing needs time.
Communications of the ACM, 52(5):70–79
DOI:10.1145/1506409.1506426

References III

[Nelson et al., 2005] Nelson, G. K., Lombardi, M. A., and Okayama, D. T. (2005).
NIST time and frequency radio stations: WWV, WWVH, and WWVB.
NIST Special Publication 250-67, National Institute of Standards and Technology
https://www.nist.gov/system/files/documents/calibrations/sp250-67.pdf

[Omicini et al., 2001] Omicini, A., Zambonelli, F., Klusch, M., and Tolksdorf, R., editors (2001).
Coordination of Internet Agents: Models, Technologies, and Applications.
Springer Berlin Heidelberg
(APICe) DOI:10.1007/978-3-662-04401-8

[Rovelli, 2017] Rovelli, C. (2017).
L'ordine del tempo, volume 705 of *Piccola Biblioteca Adelphi*.
Adelphi

[Tanenbaum and van Steen, 2017] Tanenbaum, A. S. and van Steen, M. (2017).
Distributed Systems. Principles and Paradigms.
Pearson Prentice Hall, 3rd edition
https://www.distributed-systems.net/index.php/books/ds3/

[Turing, 1937] Turing, A. M. (1937).
On computable numbers, with an application to the entscheidungsproblem.
Proceedings of the London Mathematical Society, s2-42(1):230–265
DOI:10.1112/plms/s2-42.1.230